Hochschule Konstanz
Fakultät Informatik

Software Engineering

04 - More Scala

Prof. Dr. Marko Boger

A class ...

... in Java:

```
public class Person {
  public final String name;
  public final int age;
  Person(String name, int age) {
      this.name = name;
      this.age = age;
  }
}
```

... in Scala:

```
case class Person(name: String, age: Int)
```

... and its usage

... in Java:

```
import java.util.ArrayList;
...
Person[] people;
Person[] minors;
Person[] adults;
{  ArrayList<Person> minorsList = new ArrayList<Person>();
   ArrayList<Person> adultsList = new ArrayList<Person>();
   for (int i = 0; i < people.length; i++)
       (people[i].age < 18 ? minorsList : adultsList)
       .add(people[i]);
   minors = minorsList.toArray(people);
   adults = adultsList.toArray(people);
}
```

... in Scala:

```
val people = List(Person("Marko",52))
val (minors, adults) = people partition (_.age < 18)
```

Small Grammar

The grammar of Scala 3 is very small in comparison to other languages. It fits on 8 pages in A4 format.

Here is a list of all keywords

Regular keywords

```
abstract  case      catch     class     def       do        else
enum      export    extends   false     final     finally   for
given     if        implicit  import    lazy      match     new
null      object    override  package   private   protected return
sealed    super     then      throw     trait     true      try
type      val       var       while     with      yield
:         =         <-        =>        <:        >:        #
@         =>>       ?=>
```

Soft keywords

```
as  derives  end  extension  infix  inline  opaque  open  transparent  using  |  *  +  -
```

Significant Indentation

In Scala 2, blocks of code are formed by braces, Java style

```
case class Person(name:String, birthdate:Date) extends Ordered[Person]{
 override def compare(that: Person) = this.age - that.age
 def age = birthdate.fullYearsSince
 def age(date:Date) = birthdate.fullYearsSince(date)
}
```

Scala 3 also allows blocks by significant indentation, Python style

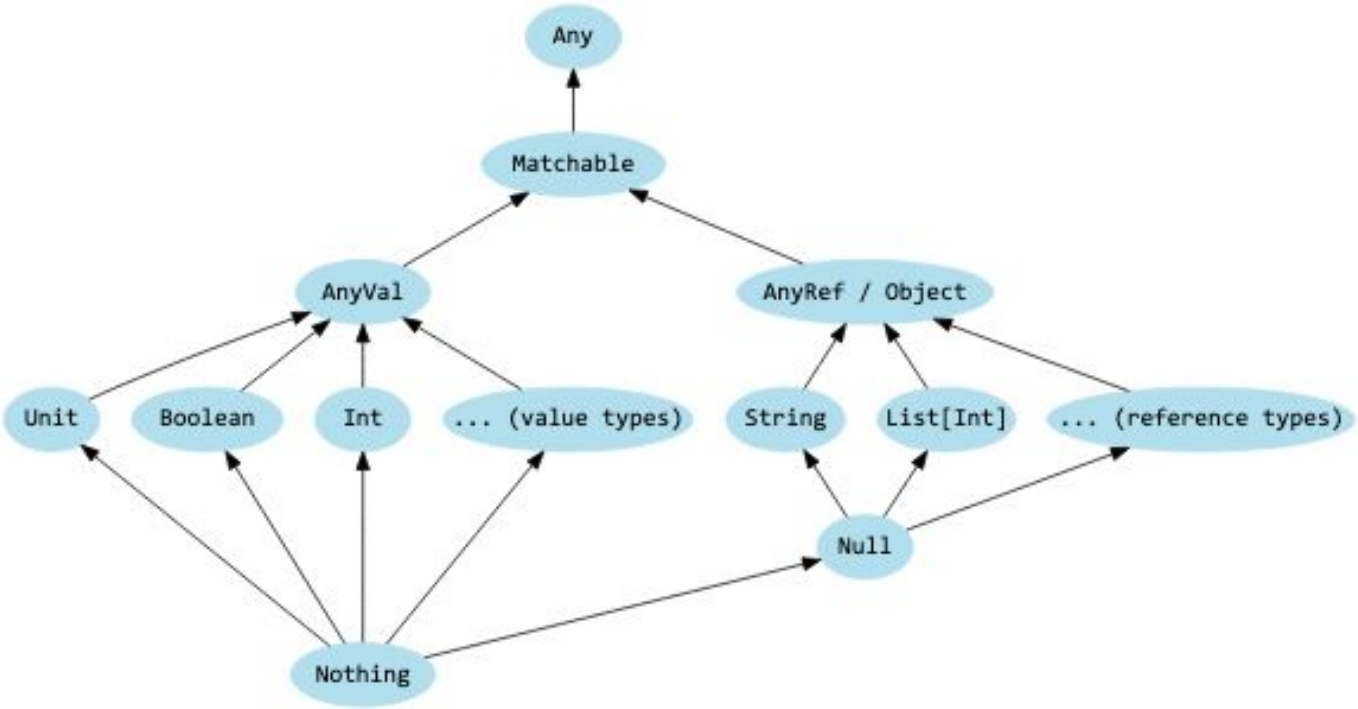
```
case class Person(name:String, birthdate:Date) extends Ordered[Person]:
 override def compare(that: Person) = this.age - that.age
 def age = birthdate.fullYearsSince
 def age(date:Date) = birthdate.fullYearsSince(date)
```

Scala Class Hierarchy

Scala has a closed type hierarchy with top and bottom types.

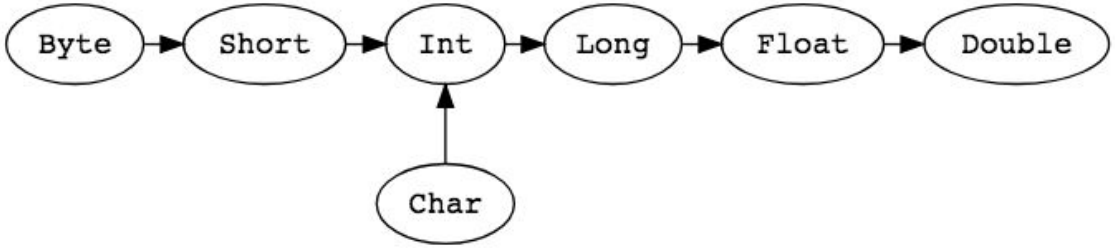
The top class is `Any`.

The bottom type is `Nothing`.



Type Casting for Value Types

Value types can easily be converted into a suitable more complex value type.



The Type Unit

- Similar to void in Java
- But Java-void is not a type, it is a keyword
- In Scala Unit is a type
  - So we can do type checking on it
  - It is a subtype of AnyVal
- It has only a single instance value
  - ()

- Try to avoid Unit, try to always return a result from a function

The Type Nothing

- Nothing is a subtype of any type
- i.e. is returned if a calculation is aborted with exception

```
def error(message:String):Nothing =
throw new RuntimeException(message)

def divide(x:Int, y:Int) : Int =
    if (y!=0) x/y
    else error(“can not divide by zero“)
```

- Both branches are compatible with type Int

The Type Null

- In Java null is a value that is allowed for many Types
  - It can be checked for its value, but not for its type
- Null in Scala is a Type
  - Null is a subtype of any Reference Type
    - It is type compatible with any Reference Type
  - null is an instance of Null
    - Value Types can not contain null, only Reference Types can

- Null should be avoided, instead the type Option should be used
  - it is only present for compatibility reasons

The Type Option

- In Java, a return value null can mean „no value found“ or dereferenced
  - It forces to check for null pointer - > Defencive Programming
- In Scala, return types that can have a value or not are modeled as Option
  - `def search(something:Any): Option[Any] = …`
- An Option can have the values
  - `Some(x)`
  - `None`
- Access this value i.e. with a match:

```
search(something) match {
    case Some(s) => println(“found “+s)
    case None => println(“found nothing“)
}
```

Extending the Type System

- Java has inheritance from one class and realization from several Interfaces
  - Interfaces can not have attributes or method implementations
  - Abstract classes can only be inherited once
    - Rich Interfaces are problematic because all methods must be provided by the user
- Scala has
  - Inheritance from a class OR a trait
  - Mix-in using ‚with‘ from several traits
  - Traits can have attributes and method implementations
  - Traits can not have constructor arguments
    - Rich Traits are very convenient, they provide a lot
- This brings Scala very close to multiple Inheritance

Traits - an Example

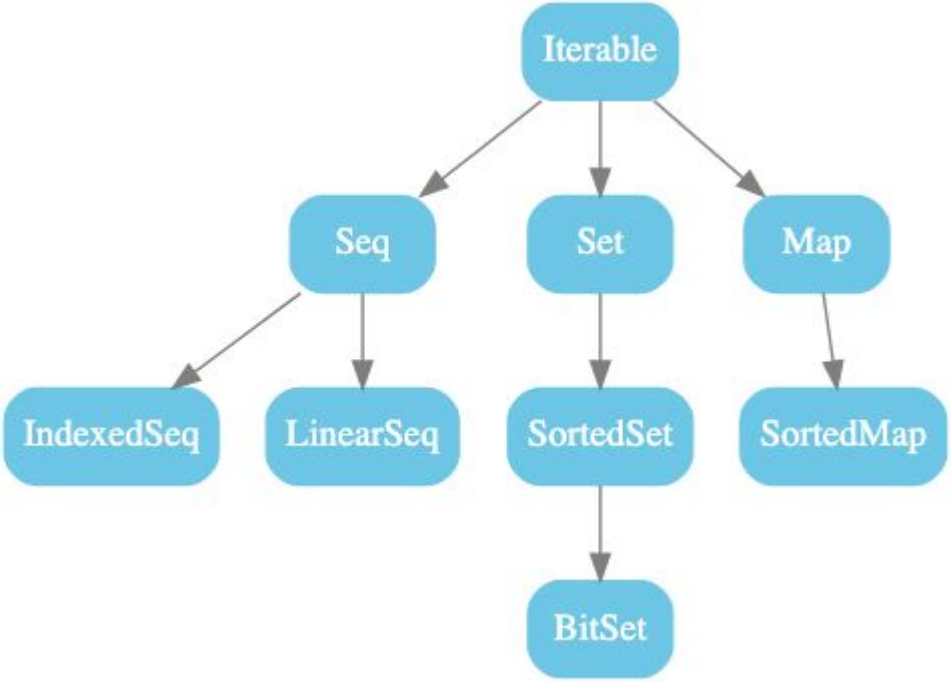
```
trait Ordered[A] {
  def compare(that: A): Int   // abstract method

  def <  (that: A): Boolean = (this compare that) <  0
  def >  (that: A): Boolean = (this compare that) >  0
}

class Health(val value : Int) extends Ordered[Health]
{
  override def compare(other : Health) = {
   this.value - other.value }
  def isCritical() = …
}
```

Collection Abstractions

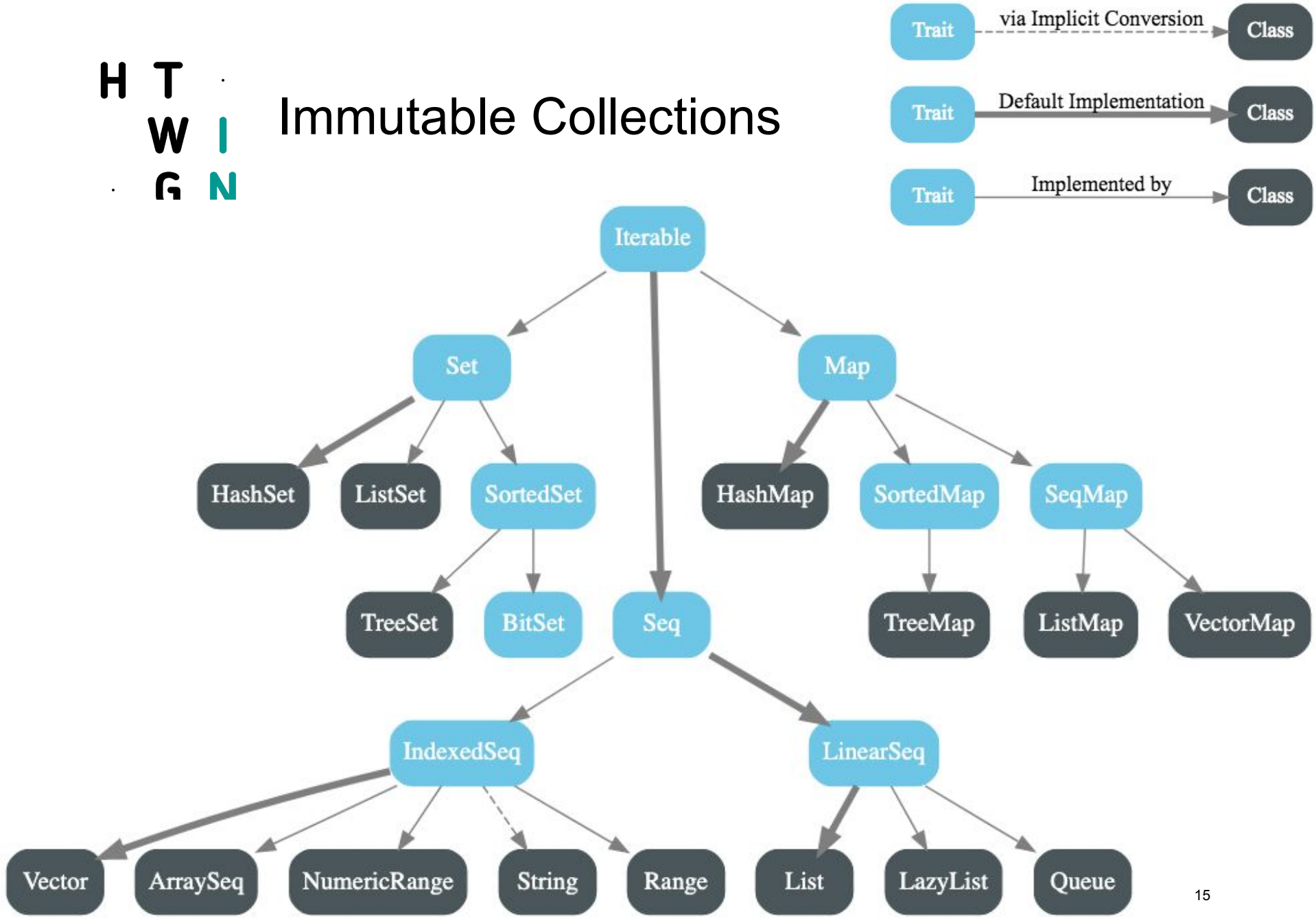
Scala has a very nice collection library. At its core are some very clean abstractions of different types of collections. They very nicely unify the API.

These abstractions are available both as immutable and mutable implementations.

Unless you have a very strong reason, use the immutable one.



Immutable Collections



Creation of Collections

Creation of collections is uniform and simple. Here are examples:

```
Iterable("x", "y", "z")
Map("x" -> 24, "y" -> 25, "z" -> 26)
Set(Color.red, Color.green, Color.blue)
SortedSet("hello", "world")
Buffer(x, y, z)
IndexedSeq(1.0, 2.0)
LinearSeq(a, b, c)
```

Often you will use the default types:

```
List(1, 2, 3)
Vector(1, 2, 3)
Map("x" -> 24, "y" -> 25, "z" -> 26)
```

Examples for Iterator

```
scala> val xs = List(1, 2, 3, 4, 5)
xs: List[Int] = List(1, 2, 3, 4, 5)
scala> val git = xs grouped 3
git: Iterator[List[Int]] = non-empty iterator
scala> git.next()
res3: List[Int] = List(1, 2, 3)
scala> git.next()
res4: List[Int] = List(4, 5)
scala> val sit = xs sliding 3
sit: Iterator[List[Int]] = non-empty iterator
scala> sit.next()
res5: List[Int] = List(1, 2, 3)
scala> sit.next()
res6: List[Int] = List(2, 3, 4)
scala> sit.next()
res7: List[Int] = List(3, 4, 5)
```

API of Set

A Set is a Traversable and an Iterator, and has specific operations for sets

`xs contains x, xs(x)`	Test whether x is an element of xs.
`xs + x`	The set containing all elements of xs as well as x.
`xs + (x, y, z)`	The set containing all elements of xs as well as with the given additional elements.
`xs ++ ys`	The set containing all elements of xs as well as all elements of ys.
`xs - x`	The set containing all elements of xs except for x.
`xs - (x, y, z)`	The set containing all elements of xs except for the given elements.
`xs - ys`	The set containing all elements of xs except for the elements of ys.
`xs & ys, xs intersect ys`	The set intersection of xs and ys.
`xs | ys, xs union ys`	The set union of xs and ys.
`xs &~ ys, xs diff ys`	The set difference of xs and ys.
`xs subsetof ys`	Test whether xs is a subset of ys.
xs.empty	An empty set of the same class as xs.

Lists

- Lists are heavily used in Scala, like in all functional programming languages
- Simple lists
  - `List(‘ a‘ , ‘ b‘ , ‘ c‘ )`
  - `List( 1, 2, 3, 4)`
  - `List(“apples“, “oranges“, “pears“)`
  - `List(List(1,0), List(0,1))`
  - `List()`
- The elements all have the same (super-) Type
  - `List[Char], List[Int], List[String], List[List[Int]]`
  - `val fruit: List[String] = List(“apples“, “oranges“)`

List concatenator

- Empty List
  - `Nil`
- Element Concatenator ::
  - `val nums = 1 :: (2 :: (3 :: (4 :: Nil)))`
- Concatenation works from right to left
  - `val nums = 1 :: 2 :: 3 :: 4 :: Nil`
- Basic Operations
  - `head`
  - `tail`
  - `isEmpty`
- List Concatenator :::
  - `List(1,2,3):::List(4,5,6)`

Tuples

Scala has a type for a sequence of elements of different type

```
(10, List('c', 'm'), "cache")
```

The type of this expression is `Tuple3[Int, List[Char], String]`

Tuples can be used to return several different types from a function.
Or to assign two values in one step.

Tuples: Example

```
def divMod(x : Int, y : Int) = (x / y, x % y)

val dm = divMod(10, 3)   // Tuple2[Int, Int]
dm._1      // 3
dm._2      // 1

val (d, m) = divMod(10, 3);
println(d + " " + m);     // 3 1
```

Control flow

Two fundamental building blocks of languages for control flow are statements and expressions.

A statement divides the control flow. It does not return a value. They have an effect, some state must be altered in one of the branches of the control flow.

An expression returns a value. Expressions are composable.

Statements always have side-effects, expressions usually don‘t.

Java is a statement-oriented language.
Scala is an expression-oriented language.

If as Statement or Expression

In Scala if can be used as statement or expression.

Here it is an expression that returns a good or bad news:

```
val age = 18

val canVoteStatus = if (age >= 18) { "You are
eligible to vote." }
else { "You are not eligible to vote." }

println(canVoteStatus)
```

And here it is a statement and does not return anything:

```
if (age >= 18) then
  println("You are eligible to vote." )
```

Control Structures

Control Structures can be written without braces and parentheses. For this to work, some need additional keyworks, like `if` needs `then`. Also, there is an optional `end` marker.

```
case class Date(year: Int = Today.year, month: Int = Today.month, day:
Int = Today.day)
     extends Ordered[Date]:
 override def compare(that: Date) =
   if this.year != that.year then
    this.year - that.year
   else if this.month != that.month then
    this.month - that.month
   else
     this.day - that.day
   end if
```

For Comprehension

For is not just a simple loop, like in (early) Java. It can iterate over complex data structures.

```
val names = Seq("chris", "ed", "maurice")

for (n <- names) println(n)
```

The argument is called a generator. It can have several generators

```
val nums = Seq(1,2,3)
val letters = Seq('a', 'b', 'c')
val res = for {
    n <- nums
    c <- letters
} yield (n, c)
```

For and Ranges

To iterate over an index, use a Range in the generator

```
for (i <- 1 to 3) println(i)
```

A Range has several nice operations

```
scala> 1 to 10
res0: scala.collection.immutable.Range.Inclusive = Range(1, 2, 3,
4, 5, 6, 7, 8, 9, 10)

scala> 1 until 10
res1: scala.collection.immutable.Range = Range(1, 2, 3, 4, 5, 6,
7, 8, 9)

scala> 1 to 10 by 2
res2: scala.collection.immutable.Range = Range(1, 3, 5, 7, 9)
```

For with Guards

Each generator can have a guard

```
for (i <- 1 to 10 if i < 4) println(i)
```

It can actually have several guards

```
for {
    i <- 1 to 10
    if i > 3
    if i < 6
    if i % 2 == 0
} println(i)
```

For with yield

A For can return a list of values, one element per iteration. To achieve this, add the keyword yield.

```
val nums = Seq(1,2,3)
val letters = Seq('a', 'b', 'c')
val res = for {
    n <- nums
    c <- letters
} yield (n, c)
res: Seq[(Int, Char)] = List((1,a), (1,b), (1,c),
                             (2,a), (2,b), (2,c),
                             (3,a), (3,b), (3,c))
```

foreach

All collections have the function foreach. It serves much like a for loop and is very common.

```
val v = Vector((1,9), (2,8), (3,7), (4,6), (5,5))

v.foreach{ case(i,j) => println(i, j) }
```

map

A typical way to express loops in FP is using map

- Map
  - Maps each element of a list to a new element of a list according to some function

```
val n = (1 to 3).toList
n.map(i => i*3)
    List[Int] = List(3, 6, 9))
n.map(i => n.map(j => i * j))
    List[List[Int]] = List(List(1, 2, 3), List(2,
    4, 6), List(3, 6, 9))
```

- Flatmap
  - Produces a not-nested list (flat)

```
n.flatMap(i => n.map(j => i * j))
    List[Int] = List(1, 2, 3, 2, 4, 6, 3, 6, )
```

match

- Match is similar to switch but more general

```
def describe(x: Int) =
  x match {
    case 1 => "one"
    case 2 => "two"
    case _ => "many"
  }
```

- Variable renaming and guards

```
def describe(x: Int) =
  x match {
    case n:Int if (n < 0) => n + " is negative"
    case z:Int if (z ==0) => z + " is zero"
    case p:Int if (p > 0) => p + " is positive"
  }
```

match on Types

The match can be used to cast types to new variables

```
def describe(x: Any) =
    x match {
      case i:Int     => i + " is an Integer"
      case s:String  => s + " is a String"
      case b:Boolean => b + " is a Boolean"
      case _         => x + " has some other type"
    }
```

Pattern Matching

- Lists can be taken apart by pattern matching

```
elem::list = 2::5::1::Nil
    elem=2, list=5::1::Nil
```

- Isort with pattern matching

```
def isort(list: List[Int]): List[Int] = list match {
  case Nil => Nil
  case head :: tail => insert(head, isort(tail))
}
def insert(elem: Int, list: List[Int]): List[Int] = list match {
  case Nil => List(elem)
  case head :: tail => if (elem <= head) elem :: list
  else head :: insert(elem, tail)
}
```

What does this Java code do?

```
public class Roman
{

    public static class SymTab
    {
        char symbol;
        long value;
        public SymTab(char s, long v) {
this.symbol=s; this.value=v; }
   };

   public static Roman.SymTab syms[]= {
        new Roman.SymTab('M',1000),
            new Roman.SymTab('D',500),
        new Roman.SymTab('C',100),
        new Roman.SymTab('L',50),
        new Roman.SymTab('X',10),
        new Roman.SymTab('V',5),
        new Roman.SymTab('I',1)
   };
```

```
public static long toLong(String s) {
        long tot=0,max=0;
        char ch[]=s.toUpperCase().toCharArray();
        int i,p;
        for(p=ch.length-1;p>=0;p--)
        {
            for(i=0;i<syms.length;i++)
            {
                if(syms[i].symbol==ch[p])
                {
                    if(syms[i].value>=max)
                        tot+= (max = syms[i].value);
                    else
                        tot-= syms[i].value;
                }
            }
        }
        return tot;
   };
}
```

An implementation in Scala using Lists

```
def roman(in: List[Char]): Int = in match {
   case 'I' :: 'V' :: rest => 4 + roman(rest)
   case 'I' :: 'X' :: rest => 9 + roman(rest)
   case 'I' :: rest => 1 + roman(rest)
   case 'V' :: rest => 5 + roman(rest)
   case 'X' :: 'L' :: rest => 40 + roman(rest)
   case 'X' :: 'C' :: rest => 90 + roman(rest)
   case 'X' :: rest => 10 + roman(rest)
   case 'L' :: rest => 50 + roman(rest)
   case 'C' :: 'D' :: rest => 400 + roman(rest)
   case 'C' :: 'M' :: rest => 900 + roman(rest)
   case 'C' :: rest => 100 + roman(rest)
   case 'D' :: rest => 500 + roman(rest)
   case 'M' :: rest => 1000 + roman(rest)
   case _ => 0
}
```

Enums

In Scala 2 enums were only possible with a somewhat complicated construct.

```
object Weekday extends Enumeration {
 val Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday =
Value
}
```

Scala 3 has introduced a clean and powerful enum. Enums follow a case and can be separated by colon. It can even have methods.

```
enum Weekdays:
 case Monday, Tuesday, Wednesday, Thursday, Friday, Saturday
 case Sunday
 def fromString(str:String) = Weekdays.valueOf(str)
```

Enums

Here is an example with Parameters and Methods

```
enum Planet(mass: Double, radius: Double) {
  private final val G = 6.67300E-11
  def surfaceGravity = G * mass / (radius * radius)
  def surfaceWeight(otherMass: Double) =  otherMass * surfaceGravity

  case Mercury extends Planet(3.303e+23, 2.4397e6)
  case Venus   extends Planet(4.869e+24, 6.0518e6)
  case Earth   extends Planet(5.976e+24, 6.37814e6)
  case Mars    extends Planet(6.421e+23, 3.3972e6)
  case Jupiter extends Planet(1.9e+27,   7.1492e7)
  case Saturn  extends Planet(5.688e+26, 6.0268e7)
  case Uranus  extends Planet(8.686e+25, 2.5559e7)
  case Neptune extends Planet(1.024e+26, 2.4746e7)
}
```

Enums with Type Classes or Abstract Data Types (ADT)

Enums can also contain a Type parameter. With this, something like Option can be constructed.

```
enum ListEnum[+A] {
  case Cons(h: A, t: ListEnum[A])
  case Empty
}

enum OptionEnum[+A] {
  case Some(content: A)
  case None
}
```

Traits with Parameter

Classes (or their constructor) can have Parameters. But Traits in Scala 2 could not. In Scala 3 now they can.

```
trait Base(val msg: String)
class A extends Base("Hello")
class B extends Base("Dotty!")
```

Union Types

Scala 3 allows a variable to have one of two (or several) types

```
case class UserName(name: String)
case class Email(address: String)
val id: UserName | Email = UserName("Marko")

case class Person(name:String, birthdate:Date) extends Ordered[Person]:
  def verify(id: UserName | Email): Boolean =
   id match
     case UserName(name) => lookupName(name)
     case Email(address) => lookupEmail(address)
```

Intersection Types

Likewise we can require a variable to have two (or more) types at the same time (usually traits). Intersection is very similar to “with” but it is commutative (the order of inheritance does not matter).

```
trait Camera:
   def takePhoto(): Unit = println("snap")
trait Phone:
   def makeCall(): Unit = println("ring")

def useSmartDevice(sp: Camera & Phone): Unit =
   sp.takePhoto()
   sp.makeCall()

class Smartphone extends Camera with Phone

useSmartDevice(new Smartphone)
```

Literal Types

1, 3.14, true and null are literals. Now they can also serve as Types.

```
val fieldSize : 1 | 4 | 9 = 9
```

ReadLine

Scala has a ReadLine, but it requires an import

```
import scala.io.StdIn.readLine
```

Then it can be used to read lines of input

```
    print("Enter your first name: ")
    val firstName = readLine()

    print("Enter your last name: ")
    val lastName = readLine()

    println(s"Your name is $firstName $lastName")
```

Task 4: Build a Text UI

Start building the core data structures of your game.
Build a simple UI using text input and output.